

monitor_1.2.py | Seite 1

```
1: from __future__ import annotations
2:
3: import inspect
4: import time
5: from dataclasses import dataclass
6:
7: import cv2
8: import numpy as np
9: import supervision as sv
10: from ultralytics import YOLO
11:
12: from .config import AppConfig, ByteTrackSettings, ZoneDefinition
13: from .esp32 import Esp32CameraClient, LatestFrameStream
14: from .zones import color_for_index, draw_polygon_overlay, scale_points
15:
16:
17: def erstelle_bytetrack(einstellungen: ByteTrackSettings) -> sv.ByteTrack:
18:     parameter = inspect.signature(sv.ByteTrack).parameters
19:     argumente: dict[str, object] = {}
20:
21:     if "track_activation_threshold" in parameter:
22:         argumente["track_activation_threshold"] = (
23:             einstellungen.track_activation_threshold
24:         )
25:     elif "track_thresh" in parameter:
26:         argumente["track_thresh"] = einstellungen.track_activation_threshold
27:
28:     if "lost_track_buffer" in parameter:
29:         argumente["lost_track_buffer"] = einstellungen.lost_track_buffer
30:     elif "track_buffer" in parameter:
31:         argumente["track_buffer"] = einstellungen.lost_track_buffer
32:
33:     if "minimum_matching_threshold" in parameter:
34:         argumente["minimum_matching_threshold"] = (
35:             einstellungen.minimum_matching_threshold
36:         )
37:     elif "match_thresh" in parameter:
38:         argumente["match_thresh"] = einstellungen.minimum_matching_threshold
39:
40:     if "frame_rate" in parameter:
41:         argumente["frame_rate"] = einstellungen.frame_rate
42:
43:     return sv.ByteTrack(**argumente)
44:
```

```
45:
46: def erstelle_polygon_zone(
47:     polygon_punkte: np.ndarray, bild_aufloesung: tuple[int, int]
48: ) -> sv.PolygonZone:
49:     parameter = inspect.signature(sv.PolygonZone).parameters
50:     argumente: dict[str, object] = {"polygon": polygon_punkte}
51:     if "frame_resolution_wh" in parameter:
52:         argumente["frame_resolution_wh"] = bild_aufloesung
53:     if "triggering_anchors" in parameter and hasattr(sv, "Position"):
54:         argumente["triggering_anchors"] = (sv.Position.BOTTOM_CENTER,)
55:     return sv.PolygonZone(**argumente)
56:
57:
58: def tracker_ids_aus_detektionen(detektionen: sv.Detections) -> set[int]:
59:     tracker_ids = getattr(detektionen, "tracker_id", None)
60:     if tracker_ids is None:
61:         return set()
62:
63:     gefundene_ids: set[int] = set()
64:     for roh_id in np.asarray(tracker_ids).reshape(-1).tolist():
65:         if roh_id is None:
66:             continue
67:         try:
68:             tracker_id = int(roh_id)
69:         except (TypeError, ValueError):
70:             continue
71:         if tracker_id >= 0:
72:             gefundene_ids.add(tracker_id)
73:     return gefundene_ids
74:
75:
76: @dataclass
77: class ZwischengespeicherteZone:
78:     definition: ZoneDefinition
79:     polygon: np.ndarray
80:     zone: sv.PolygonZone
81:
82:
83: @dataclass
84: class FrameZeiten:
85:     quell_ms: float = 0.0
86:     inferenz_ms: float = 0.0
87:     tracking_ms: float = 0.0
88:     zeichnen_ms: float = 0.0
```

monitor_1.2.py | Seite 3

```
89:     gesamt_ms: float = 0.0
90:
91:
92: class AlarmSteuerung:
93:     def __init__(self, kamera_client: Esp32CameraClient, config: AppConfig) -> None:
94:         self.kamera_client = kamera_client
95:         self.config = config
96:         self.letzte_ausloesung = 0.0
97:         self.letzte_meldung = ""
98:         self.letzter_fehler = ""
99:
100:     def ausloesen(self, zonen_name: str, tracker_ids: list[int]) -> bool:
101:         jetzt = time.monotonic()
102:         if jetzt - self.letzte_ausloesung < self.config.alarm.cooldown_seconds:
103:             return False
104:
105:         self.letzte_ausloesung = jetzt
106:         ids_text = ", ".join(str(tracker_id) for tracker_id in tracker_ids)
107:         self.letzte_meldung = f"ALARM: {zonen_name} betreten von Track(s) {ids_text}"
108:         self.letzter_fehler = ""
109:
110:         print(f"[ALARM] {self.letzte_meldung}")
111:         if self.config.alarm.local_terminal_bell:
112:             print("\a", end="", flush=True)
113:
114:         try:
115:             self.kamera_client.pulse_buzzer(self.config.alarm.pulse_ms)
116:         except Exception as fehler:
117:             self.letzter_fehler = str(fehler)
118:             print(f"[WARN] ESP32-Buzzer konnte nicht ausgelost werden: {fehler}")
119:
120:         return True
121:
122:     def banner_text(self) -> str:
123:         halte_zeit = max(2.0, self.config.alarm.pulse_ms / 1000.0)
124:         if time.monotonic() - self.letzte_ausloesung <= halte_zeit:
125:             return self.letzte_meldung
126:         return ""
127:
128:
129: class UeberwachungsMonitor:
130:     def __init__(self, config: AppConfig) -> None:
131:         self.config = config
132:         self.kamera_client = Esp32CameraClient(config)
```

```
133:         self.modell = YOLO(config.model)
134:         self.alarm = AlarmSteuerung(self.kamera_client, config)
135:         self._zonen_cache_aufloesung: tuple[int, int] | None = None
136:         self._zonen_cache: list[ZwischengespeicherteZone] = []
137:         self._fps = 0.0
138:         self._letztes_frame_zeit = 0.0
139:         self._zeiten = FrameZeiten()
140:         self.laufzeit_zuruecksetzen()
141:
142:     def _zeit_aktualisieren(self, name: str, wert_ms: float) -> None:
143:         bisheriger_wert = getattr(self._zeiten, name)
144:         neuer_wert = wert_ms if bisheriger_wert == 0.0 else bisheriger_wert * 0.8 + wert_ms * 0.2
145:         setattr(self._zeiten, name, neuer_wert)
146:
147:     def laufzeit_zuruecksetzen(self) -> None:
148:         self.tracker = erstelle_bytetrack(self.config.byetrack)
149:         self.vorherige_zonen_mitglieder = {
150:             zone.id: set() for zone in self.config.zones
151:         }
152:         self._zonen_cache_aufloesung = None
153:         self._zonen_cache = []
154:
155:     def ausfuehren(self) -> None:
156:         if not self.config.zones:
157:             raise RuntimeError("Keine Zonen konfiguriert. Zuerst draw-zones ausfuehren.")
158:
159:         if self.config.source_mode == "snapshot":
160:             self.snapshot_modus_ausfuehren()
161:             return
162:
163:         self.stream_modus_ausfuehren()
164:
165:     def stream_modus_ausfuehren(self) -> None:
166:         stream_leser = LatestFrameStream(self.kamera_client)
167:         letzte_sequenz = 0
168:         letzte_warnung_zeit = 0.0
169:
170:         print("[INFO] Source mode: stream")
171:         print(f"[INFO] Stream URL: {self.config.stream_url}")
172:         print(f"[INFO] Model: {self.config.model}")
173:         print("[INFO] Press Q or ESC in the preview window to stop.")
174:
175:         self.laufzeit_zuruecksetzen()
176:         stream_leser.start()
```

```
177:
178:     try:
179:         while True:
180:             letzte_sequenz, frame = stream_leser.read_latest(letzte_sequenz)
181:             if frame is None:
182:                 if (
183:                     stream_leser.last_error
184:                     and time.monotonic() - letzte_warnung_zeit > 1.0
185:                 ):
186:                     print(f"[WARN] {stream_leser.last_error}")
187:                     letzte_warnung_zeit = time.monotonic()
188:             if self.config.show_window:
189:                 taste = cv2.waitKey(1) & 0xFF
190:                 if taste in (27, ord("q")):
191:                     break
192:                 time.sleep(0.005)
193:                 continue
194:
195:             self.zeit_aktualisieren("quell_ms", stream_leser.source_read_ms)
196:             annotiertes_bild = self.frame_verarbeiten(frame)
197:
198:             if self.config.show_window:
199:                 cv2.imshow(self.config.window_name, annotiertes_bild)
200:                 taste = cv2.waitKey(1) & 0xFF
201:                 if taste in (27, ord("q")):
202:                     break
203:         finally:
204:             stream_leser.stop()
205:             cv2.destroyAllWindows()
206:
207:     def snapshot_modus_ausfuehren(self) -> None:
208:         intervall_sekunden = max(0.05, self.config.snapshot_interval_ms / 1000.0)
209:
210:         print("[INFO] Source mode: snapshot")
211:         print(f"[INFO] Capture URL: {self.config.capture_url}")
212:         print(f"[INFO] Model: {self.config.model}")
213:         print("[INFO] Press Q or ESC in the preview window to stop.")
214:
215:         self.laufzeit_zuruecksetzen()
216:
217:     try:
218:         while True:
219:             schleifen_start = time.monotonic()
220:             quell_start = time.perf_counter()
```

```
221:         try:
222:             frame = self.kamera_client.capture_frame()
223:         except Exception as fehler:
224:             print(f"[WARN] Snapshot fehlgeschlagen: {fehler}")
225:             time.sleep(1.0)
226:             continue
227:         self._zeit_aktualisieren(
228:             "quell_ms", (time.perf_counter() - quell_start) * 1000.0
229:         )
230:
231:         annotiertes_bild = self.frame_verarbeiten(frame)
232:
233:         if self.config.show_window:
234:             cv2.imshow(self.config.window_name, annotiertes_bild)
235:             taste = cv2.waitKey(1) & 0xFF
236:             if taste in (27, ord("q")):
237:                 break
238:
239:         vergangene_zeit = time.monotonic() - schleifen_start
240:         restzeit = intervall_sekunden - vergangene_zeit
241:         if restzeit > 0:
242:             time.sleep(restzeit)
243:     finally:
244:         cv2.destroyAllWindows()
245:
246:     def frame_verarbeiten(self, frame: np.ndarray) -> np.ndarray:
247:         gesamt_start = time.perf_counter()
248:         jetzt = time.monotonic()
249:         if self._letztes_frame_zeit:
250:             delta = jetzt - self._letztes_frame_zeit
251:             if delta > 0:
252:                 aktuelle_fps = 1.0 / delta
253:                 if self._fps == 0.0:
254:                     self._fps = aktuelle_fps
255:                 else:
256:                     self._fps = self._fps * 0.9 + aktuelle_fps * 0.1
257:         self._letztes_frame_zeit = jetzt
258:
259:         inferenz_start = time.perf_counter()
260:         detektionen = self.personen_erkennen(frame)
261:         self._zeit_aktualisieren(
262:             "inferenz_ms", (time.perf_counter() - inferenz_start) * 1000.0
263:         )
264:
```

monitor_1.2.py | Seite 7

```
265:         tracking_start = time.perf_counter()
266:         getrackte_objekte = self.tracker.update_with_detections(detektionen)
267:         laufzeit_zonen = self.laufzeit_zonen(frame)
268:
269:         verletzende_tracker_ids: set[int] = set()
270:         for zwischengespeicherte_zone in laufzeit_zonen:
271:             maske = zwischengespeicherte_zone.zone.trigger(getrackte_objekte)
272:             objekte_in_zone = getrackte_objekte[maske]
273:             aktuelle_ids = tracker_ids_aus_detektionen(objekte_in_zone)
274:             verletzende_tracker_ids.update(aktuelle_ids)
275:
276:             vorherige_ids = self.vorherige_zonen_mitglieder.setdefault(
277:                 zwischengespeicherte_zone.definition.id, set()
278:             )
279:             neu_eingetretene_ids = sorted(aktuelle_ids - vorherige_ids)
280:             self.vorherige_zonen_mitglieder[
281:                 zwischengespeicherte_zone.definition.id
282:             ] = aktuelle_ids
283:
284:             if neu_eingetretene_ids:
285:                 self.alarm.ausloesen(
286:                     zwischengespeicherte_zone.definition.name,
287:                     neu_eingetretene_ids,
288:                 )
289:
290:         self._zeit_aktualisieren(
291:             "tracking_ms", (time.perf_counter() - tracking_start) * 1000.0
292:         )
293:
294:         zeichnen_start = time.perf_counter()
295:         annotiertes_bild = self.frame_annotieren(
296:             frame,
297:             getrackte_objekte,
298:             laufzeit_zonen,
299:             verletzende_tracker_ids,
300:         )
301:         self._zeit_aktualisieren(
302:             "zeichnen_ms", (time.perf_counter() - zeichnen_start) * 1000.0
303:         )
304:         self._zeit_aktualisieren(
305:             "gesamt_ms", (time.perf_counter() - gesamt_start) * 1000.0
306:         )
307:
308:         return annotiertes_bild
```

```

309:
310:     def personen_erkennen(self, frame: np.ndarray) -> sv.Detections:
311:         vorhersage_argumente = {
312:             "source": frame,
313:             "conf": self.config.confidence,
314:             "iou": self.config.iou_threshold,
315:             "classes": [0],
316:             "imgsz": self.config.image_size,
317:             "verbose": False,
318:         }
319:         if self.config.device:
320:             vorhersage_argumente["device"] = self.config.device
321:
322:         ergebnis = self.modell.predict(**vorhersage_argumente)[0]
323:         detektionen = sv.Detections.from_ultralytics(ergebnis)
324:         if len(detektionen) == 0:
325:             return detektionen
326:
327:         if getattr(detektionen, "class_id", None) is None:
328:             return detektionen
329:
330:         return detektionen[detektionen.class_id == 0]
331:
332:     def laufzeit_zonen(self, frame: np.ndarray) -> list[ZwischengespeicherteZone]:
333:         aktuelle_aufloesung = (int(frame.shape[1]), int(frame.shape[0]))
334:         if self._zonen_cache_aufloesung == aktuelle_aufloesung:
335:             return self._zonen_cache
336:
337:         neue_zonen: list[ZwischengespeicherteZone] = []
338:         for zone in self.config.zones:
339:             skaliertes_polygon = scale_points(
340:                 zone.points,
341:                 self.config.zone_resolution,
342:                 aktuelle_aufloesung[0],
343:                 aktuelle_aufloesung[1],
344:             )
345:             neue_zonen.append(
346:                 ZwischengespeicherteZone(
347:                     definition=zone,
348:                     polygon=skaliertes_polygon,
349:                     zone=erstelle_polygon_zone(
350:                         skaliertes_polygon, aktuelle_aufloesung
351:                     ),
352:                 )

```



```

353:         )
354:
355:         self._zonen_cache_aufloesung = aktuelle_aufloesung
356:         self._zonen_cache = neue_zonen
357:         return neue_zonen
358:
359:     def frame_annotieren(
360:         self,
361:         frame: np.ndarray,
362:         getrackte_objekte: sv.Detections,
363:         laufzeit_zonen: list[ZwischengespeicherteZone],
364:         verletzende_tracker_ids: set[int],
365:     ) -> np.ndarray:
366:         annotiertes_bild = frame.copy()
367:
368:         for index, zwischengespeicherte_zone in enumerate(laufzeit_zonen):
369:             aktuelle_mitglieder = self.vorherige_zonen_mitglieder.get(
370:                 zwischengespeicherte_zone.definition.id, set()
371:             )
372:             annotiertes_bild = draw_polygon_overlay(
373:                 annotiertes_bild,
374:                 zwischengespeicherte_zone.polygon,
375:                 color_for_index(index),
376:                 f"{zwischengespeicherte_zone.definition.name}: {len(aktuelle_mitglieder)}",
377:                 is_alert=bool(aktuelle_mitglieder),
378:             )
379:
380:         tracker_ids = getattr(getrackte_objekte, "tracker_id", None)
381:         confidences = getattr(getrackte_objekte, "confidence", None)
382:
383:         for index, box in enumerate(getrackte_objekte.xyxy):
384:             x1, y1, x2, y2 = [int(wert) for wert in box]
385:             tracker_id = None
386:             if tracker_ids is not None:
387:                 rohe_id = tracker_ids[index]
388:                 if rohe_id is not None:
389:                     tracker_id = int(rohe_id)
390:
391:             sicherheit = 0.0
392:             if confidences is not None:
393:                 sicherheit = float(confidences[index])
394:
395:             ist_verletzend = (
396:                 tracker_id is not None and tracker_id in verletzende_tracker_ids

```

```
397:         )
398:         farbe = (0, 0, 255) if ist_verletzend else (0, 220, 0)
399:
400:         cv2.rectangle(
401:             annotiertes_bild, (x1, y1), (x2, y2), farbe, 2, cv2.LINE_AA
402:         )
403:         beschriftung = (
404:             f"person #{tracker_id} {sicherheit:.2f}"
405:             if tracker_id is not None
406:             else f"person {sicherheit:.2f}"
407:         )
408:         cv2.putText(
409:             annotiertes_bild,
410:             beschriftung,
411:             (x1, max(24, y1 - 10)),
412:             cv2.FONT_HERSHEY_SIMPLEX,
413:             0.6,
414:             farbe,
415:             2,
416:             cv2.LINE_AA,
417:         )
418:
419:         return annotiertes_bild
420:
421:
422: VisionMonitor = UeberwachungsMonitor
```

```
1: from __future__ import annotations
2:
3: import threading
4: import time
5: from typing import Any
6:
7: import cv2
8: import numpy as np
9: import requests
10:
11: from .config import AppConfig
12:
13:
14: class Esp32KameraClient:
15:     def __init__(self, config: AppConfig) -> None:
16:         self.config = config
17:
18:     def frame_aufnehmen(self) -> np.ndarray:
19:         antwort = requests.get(self.config.capture_url, timeout=10)
20:         antwort.raise_for_status()
21:
22:         frame = cv2.imdecode(
23:             np.frombuffer(antwort.content, dtype=np.uint8), cv2.IMREAD_COLOR
24:         )
25:         if frame is None:
26:             raise RuntimeError(
27:                 f"JPEG von {self.config.capture_url} konnte nicht dekodiert werden."
28:             )
29:         return frame
30:
31:     def stream_oeffnen(self) -> Any:
32:         backends = []
33:         ffmpeg_backend = getattr(cv2, "CAP_FFMPEG", None)
34:         if ffmpeg_backend is not None:
35:             backends.append(ffmpeg_backend)
36:         backends.append(None)
37:
38:         for backend in backends:
39:             if backend is None:
40:                 video_capture = cv2.VideoCapture(self.config.stream_url)
41:             else:
42:                 video_capture = cv2.VideoCapture(self.config.stream_url, backend)
43:
44:             video_capture.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

esp32_1.0.py | Seite 2

```
45:         if video_capture.isOpened():
46:             return video_capture
47:
48:         video_capture.release()
49:
50:     return cv2.VideoCapture(self.config.stream_url)
51:
52:     def buzzer_impuls(self, dauer_ms: int) -> None:
53:         parameter = {"state": "on"}
54:         if dauer_ms > 0:
55:             parameter["duration_ms"] = int(dauer_ms)
56:
57:         antwort = requests.get(self.config.buzzer_url, params=parameter, timeout=2)
58:         antwort.raise_for_status()
59:
60:     def buzzer_stoppen(self) -> None:
61:         antwort = requests.get(
62:             self.config.buzzer_url, params={"state": "off"}, timeout=2
63:         )
64:         antwort.raise_for_status()
65:
66:
67: class NeuestesFrameStream:
68:     def __init__(self, kamera_client: Esp32KameraClient) -> None:
69:         self.kamera_client = kamera_client
70:         self._sperre = threading.Lock()
71:         self._stop_ereignis = threading.Event()
72:         self._thread: threading.Thread | None = None
73:         self._capture: Any = None
74:         self._neuestes_frame: np.ndarray | None = None
75:         self._neueste_sequenz = 0
76:         self.letzter_fehler = ""
77:         self.quell lese_ms = 0.0
78:
79:     def start(self) -> None:
80:         if self._thread is not None:
81:             return
82:         self._thread = threading.Thread(target=self._lauf, daemon=True)
83:         self._thread.start()
84:
85:     def _lauf(self) -> None:
86:         aufeinanderfolgende_fehler = 0
87:
88:         while not self._stop_ereignis.is_set():
```

```

89:         if self._capture is None or not self._capture.isOpened():
90:             self._capture = self.kamera_client.stream_oeffnen()
91:             if not self._capture.isOpened():
92:                 self.letzter_fehler = (
93:                     f"{self.kamera_client.config.stream_url} konnte nicht geoffnet werden"
94:                 )
95:                 time.sleep(1.0)
96:                 continue
97:             aufeinanderfolgende_fehler = 0
98:
99:         lese_start = time.perf_counter()
100:        erfolgreich, frame = self._capture.read()
101:        lese_ms = (time.perf_counter() - lese_start) * 1000.0
102:        self.quell_lese_ms = (
103:            lese_ms
104:            if self.quell_lese_ms == 0.0
105:            else self.quell_lese_ms * 0.8 + lese_ms * 0.2
106:        )
107:
108:        if not erfolgreich or frame is None:
109:            aufeinanderfolgende_fehler += 1
110:            if aufeinanderfolgende_fehler >= 10:
111:                self.letzter_fehler = "Stream hangt. Verbindung wird neu aufgebaut..."
112:                self._capture.release()
113:                self._capture = None
114:                time.sleep(0.25)
115:                continue
116:
117:            aufeinanderfolgende_fehler = 0
118:            self.letzter_fehler = ""
119:            with self._sperr:
120:                self._neuestes_frame = frame
121:                self._neueste_sequenz += 1
122:
123:        def neuestes_lesen(
124:            self, letzte_sequenz: int
125:        ) -> tuple[int, np.ndarray | None]:
126:            with self._sperr:
127:                if (
128:                    self._neuestes_frame is None
129:                    or self._neueste_sequenz == letzte_sequenz
130:                ):
131:                    return letzte_sequenz, None
132:            return self._neueste_sequenz, self._neuestes_frame.copy()

```

```
133:
134:     def stoppen(self) -> None:
135:         self._stop_ereignis.set()
136:         if self._thread is not None:
137:             self._thread.join(timeout=2.0)
138:         if self._capture is not None:
139:             self._capture.release()
140:
141:
142: Esp32CameraClient = Esp32KameraClient
143: LatestFrameStream = NeuestesFrameStream
```

```
1: #include "esp_camera.h"
2: #include "esp_wifi.h"
3: #include "freertos/FreeRTOS.h"
4: #include "freertos/semphr.h"
5: #include <stdarg.h>
6: #include <WiFi.h>
7:
8: #ifndef BUZZER_PIN
9: #define BUZZER_PIN 2
10: #endif
11:
12: // =====
13: // WLAN-ZUGANG
14: // =====
15: const char *wlan_name = "FRITZ!Box 7530 NV";
16: const char *wlan_passwort = "Donner1997";
17:
18: // =====
19: // PINBELEGUNG FUER FREENOVE ESP32-S3 (0V3660)
20: // =====
21: #define PWDN_GPIO_NUM -1
22: #define RESET_GPIO_NUM -1
23: #define XCLK_GPIO_NUM 15
24: #define SIOD_GPIO_NUM 4
25: #define SIOC_GPIO_NUM 5
26:
27: #define Y9_GPIO_NUM 16
28: #define Y8_GPIO_NUM 17
29: #define Y7_GPIO_NUM 18
30: #define Y6_GPIO_NUM 12
31: #define Y5_GPIO_NUM 10
32: #define Y4_GPIO_NUM 8
33: #define Y3_GPIO_NUM 9
34: #define Y2_GPIO_NUM 11
35:
36: #define VSYNC_GPIO_NUM 6
37: #define HREF_GPIO_NUM 7
38: #define PCLK_GPIO_NUM 13
39:
40: // Wird aus app_httpd.cpp bereitgestellt.
41: void startCameraServer();
42:
43: // Diese Namen bleiben gleich, weil app_httpd.cpp sie per extern erwartet.
44: volatile uint32_t wifiDisconnectCount = 0;
```

```
45: volatile uint32_t wifiGotIpCount = 0;
46: volatile uint32_t buzzerPulseUntilMs = 0;
47:
48: static SemaphoreHandle_t fern_log_mutex = NULL;
49: static String fern_log_puffer;
50: static const size_t FERN_LOG_MAX_BYTES = 16384;
51:
52: static void fern_log_anhaengen_gesperrt(const char *nachricht) {
53:     fern_log_puffer += nachricht;
54:     size_t ueberlauf =
55:         fern_log_puffer.length() > FERN_LOG_MAX_BYTES
56:         ? fern_log_puffer.length() - FERN_LOG_MAX_BYTES
57:         : 0;
58:     if (ueberlauf > 0) {
59:         fern_log_puffer.remove(0, ueberlauf);
60:     }
61: }
62:
63: void fern_log_puffer_initialisieren() {
64:     if (fern_log_mutex == NULL) {
65:         fern_log_mutex = xSemaphoreCreateMutex();
66:     }
67: }
68:
69: // Namen bleiben gleich fuer die Kopplung mit app_httpd.cpp.
70: void remoteLogf(const char *format, ...) {
71:     char nachricht[256];
72:     va_list argumente;
73:     va_start(argumente, format);
74:     vsnprintf(nachricht, sizeof(nachricht), format, argumente);
75:     va_end(argumente);
76:
77:     Serial.print(nachricht);
78:
79:     if (fern_log_mutex != NULL &&
80:         xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
81:         fern_log_anhaengen_gesperrt(nachricht);
82:         xSemaphoreGive(fern_log_mutex);
83:     }
84: }
85:
86: void remoteLogln(const char *nachricht) {
87:     Serial.println(nachricht);
88:
```



```
89:   if (fern_log_mutex != NULL &&
90:       xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
91:       fern_log_anhaengen_gesperrt(nachricht);
92:       fern_log_anhaengen_gesperrt("\n");
93:       xSemaphoreGive(fern_log_mutex);
94:   }
95: }
96:
97: String getRemoteLogSnapshot(bool nach_lesen_leeren) {
98:   String schnappschuss;
99:   if (fern_log_mutex != NULL &&
100:      xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
101:     schnappschuss = fern_log_puffer;
102:     if (nach_lesen_leeren) {
103:       fern_log_puffer = "";
104:     }
105:     xSemaphoreGive(fern_log_mutex);
106:   }
107:   return schnappschuss;
108: }
109:
110: void bei_wlan_ereignis(WiFiEvent_t ereignis, WiFiEventInfo_t info) {
111:   switch (ereignis) {
112:     case ARDUINO_EVENT_WIFI_STA_DISCONNECTED:
113:       wifiDisconnectCount++;
114:       remoteLogf("[WIFI] disconnected reason=%u total=%lu\n",
115:                  info.wifi_sta_disconnected.reason,
116:                  (unsigned long)wifiDisconnectCount);
117:       break;
118:     case ARDUINO_EVENT_WIFI_STA_GOT_IP:
119:       wifiGotIpCount++;
120:       remoteLogf("[WIFI] got_ip total=%lu ip=%s\n",
121:                  (unsigned long)wifiGotIpCount,
122:                  WiFi.localIP().toString().c_str());
123:       break;
124:     default:
125:       break;
126:   }
127: }
128:
129: void setup() {
130:   Serial.begin(115200);
131:   Serial.setDebugOutput(false);
132:   fern_log_puffer_initialisieren();
```

```
133:   Serial.println();
134:
135:   pinMode(BUZZER_PIN, OUTPUT);
136:   digitalWrite(BUZZER_PIN, LOW);
137:
138:   camera_config_t kamera_konfiguration;
139:   kamera_konfiguration.ledc_channel = LEDC_CHANNEL_0;
140:   kamera_konfiguration.ledc_timer = LEDC_TIMER_0;
141:   kamera_konfiguration.pin_d0 = Y2_GPIO_NUM;
142:   kamera_konfiguration.pin_d1 = Y3_GPIO_NUM;
143:   kamera_konfiguration.pin_d2 = Y4_GPIO_NUM;
144:   kamera_konfiguration.pin_d3 = Y5_GPIO_NUM;
145:   kamera_konfiguration.pin_d4 = Y6_GPIO_NUM;
146:   kamera_konfiguration.pin_d5 = Y7_GPIO_NUM;
147:   kamera_konfiguration.pin_d6 = Y8_GPIO_NUM;
148:   kamera_konfiguration.pin_d7 = Y9_GPIO_NUM;
149:   kamera_konfiguration.pin_xclk = XCLK_GPIO_NUM;
150:   kamera_konfiguration.pin_pclk = PCLK_GPIO_NUM;
151:   kamera_konfiguration.pin_vsync = VSYNC_GPIO_NUM;
152:   kamera_konfiguration.pin_href = HREF_GPIO_NUM;
153:   kamera_konfiguration.pin_sccb_sda = SIOD_GPIO_NUM;
154:   kamera_konfiguration.pin_sccb_scl = SIOC_GPIO_NUM;
155:   kamera_konfiguration.pin_pwdn = PWDN_GPIO_NUM;
156:   kamera_konfiguration.pin_reset = RESET_GPIO_NUM;
157:   kamera_konfiguration.xclk_freq_hz = 15000000;
158:   kamera_konfiguration.frame_size = FRAMESIZE_VGA;
159:   kamera_konfiguration.pixel_format = PIXFORMAT_JPEG;
160:   kamera_konfiguration.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
161:   kamera_konfiguration.fb_location = CAMERA_FB_IN_PSRAM;
162:   kamera_konfiguration.jpeg_quality = 16;
163:   kamera_konfiguration.fb_count = 1;
164:
165:   if (psramFound()) {
166:     kamera_konfiguration.jpeg_quality = 14;
167:     kamera_konfiguration.fb_count = 2;
168:     kamera_konfiguration.frame_size = FRAMESIZE_VGA;
169:     kamera_konfiguration.grab_mode = CAMERA_GRAB_LATEST;
170:     remoteLogln("psramFound");
171:   } else {
172:     kamera_konfiguration.frame_size = FRAMESIZE_VGA;
173:     kamera_konfiguration.fb_location = CAMERA_FB_IN_DRAM;
174:   }
175:
176:   esp_err_t fehler = esp_camera_init(&kamera_konfiguration);
```

```
177:   if (fehler != ESP_OK) {
178:       remoteLogf("Kamera-Init fehlgeschlagen: 0x%x\n", fehler);
179:       return;
180:   }
181:
182:   sensor_t *kamera_sensor = esp_camera_sensor_get();
183:   if (kamera_sensor->id.PID == OV3660_PID) {
184:       kamera_sensor->set_vflip(kamera_sensor, 1);
185:       kamera_sensor->set_brightness(kamera_sensor, 1);
186:       kamera_sensor->set_saturation(kamera_sensor, -2);
187:   }
188:
189:   WiFi.onEvent(bei_wlan_ereignis);
190:   WiFi.mode(WIFI_STA);
191:   WiFi.begin(wlan_name, wlan_passwort);
192:   WiFi.setSleep(false);
193:
194:   while (WiFi.status() != WL_CONNECTED) {
195:       delay(500);
196:       Serial.print(".");
197:   }
198:   remoteLogln("WiFi verbunden!");
199:   esp_wifi_set_ps(WIFI_PS_NONE);
200:   remoteLogln("WLAN Power Management deaktiviert fuer stabilen Stream");
201:
202:   startCameraServer();
203:
204:   Serial.print("Kamera bereit! Oeffne: http://");
205:   Serial.println(WiFi.localIP());
206: }
207:
208: void loop() {
209:   uint32_t buzzer_ende_zeit = buzzerPulseUntilMs;
210:   if (buzzer_ende_zeit != 0 && (int32_t)(millis() - buzzer_ende_zeit) >= 0) {
211:       digitalWrite(BUZZER_PIN, LOW);
212:       buzzerPulseUntilMs = 0;
213:   }
214:
215:   delay(5);
216: }
```